EXPERIMENT 9

AIM

Create an ERC 20 Token in Remix IDE.

THEORY

1.1 What is a Token?

A token is a digital asset created and managed using a blockchain's smart-contract functionality. Unlike the native cryptocurrency of a blockchain, a token is generally created by a smart contract according to a predefined token standard. On Ethereum, tokens can follow standards such as ERC-20 (fungible tokens), ERC-721 (non-fungible tokens, NFTs) and ERC-1155 (supports both fungible and non-fungible assets). An ERC-20 token represents a fungible asset, meaning every unit of the token is interchangeable with another unit of the same token: 1 ABC token = 1 ABC token. Each token has the same value and properties as another token of the same type.

1.2 What is ERC-20?

ERC-20 stands for Ethereum Request for Comment 20. It defines a standard interface for fungible tokens on Ethereum-compatible blockchains. The standard specifies functions and events that allow wallets, exchanges and decentralized applications to interact with tokens in a common way. An ERC-20 token maintains information such as the token name, token symbol, total supply, balances of individual addresses, and allowances given to other addresses.

1.3 Remix IDE

Remix IDE is a browser-based development environment used for writing, compiling, deploying and testing Ethereum smart contracts written in Solidity. The major components used in this experiment are: Solidity → Remix Compiler → Remix Deploy & Run → MetaMask → Sepolia → Ethereum Blockchain. Remix allows us to write Solidity code, compile the smart contract, select an environment for deployment, connect to MetaMask, deploy the contract, execute contract functions, and view transaction results.

1.4 MetaMask

MetaMask is a cryptocurrency wallet that manages Ethereum accounts and allows users to interact with decentralized applications and smart contracts. In this experiment, MetaMask is used to create/manage the Ethereum account, connect Remix with the wallet, select the Sepolia test network, hold test ETH, approve deployment transactions, pay gas fees, and interact with the deployed ERC-20 contract.

1.5 Sepolia Testnet

Sepolia is an Ethereum test network used for development and testing. Instead of real ETH, the network uses test ETH, which has no real monetary value. Test ETH is required because deploying and interacting with a smart contract consumes gas. The basic flow is: Sepolia Faucet → Test ETH → MetaMask → Remix → Smart Contract Deployment.

2. ERC-20 Token Functions

name() - Returns the name of the token

symbol() - Returns the token symbol
decimals() - Returns the number of decimal places
totalSupply() - Returns total token supply
balanceOf(address) - Returns the token balance of an address
transfer(address, amount) - Transfers tokens to another address
approve(address, amount) - Gives another address permission to spend tokens
allowance(owner, spender) - Returns the amount a spender is allowed to spend
transferFrom(owner, recipient, amount) - Transfers tokens using an approved allowance

3. ERC-20 Events

Events are emitted by smart contracts to record important activities that can be observed by blockchain applications. The Transfer event - event Transfer(address indexed from, address indexed to, uint256 value) - is emitted whenever tokens are transferred. The Approval event - event Approval(address indexed owner, address indexed spender, uint256 value) - is emitted when a token owner approves another address to spend tokens.

4. ERC-20 Attributes

Important ERC-20 token attributes include: Name (the human-readable name of the token, e.g. College Token), Symbol (the short representation, e.g. CLG), Decimals (how many decimal places the token supports; the commonly used value is 18), Total Supply (the total number of tokens currently existing), and Balance (the number of tokens held by a particular Ethereum address).

5. ERC-20 vs ERC-721

ERC-20 tokens are fungible: tokens are interchangeable, units are generally identical, they can represent fractional units, and an address holds a quantity - examples are USDT and DAI, used as currency and utility/governance tokens. ERC-721 tokens are non-fungible: each token is unique with its own token ID, an individual NFT is generally treated as a unique asset, and ownership is associated with specific token IDs - used for digital art, collectibles and ownership.

Mintable, Burnable, Pausable

Minting means creating new tokens, which increases the total token supply (supply 1,000, mint 500, new supply 1,500). Burning means permanently destroying tokens, which decreases the total supply (supply 1,500, burn 200, new supply 1,300). The Pausable feature allows authorized accounts to temporarily stop token transfers or other selected operations, which can be useful in emergencies such as a security vulnerability, suspicious activity, or a smart-contract issue; once unpaused, normal operations resume.

CODE AND OUTPUT

Smart Contract - MyERC20Token.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/*
 * ============================================================
 * Experiment 9 : Create an ERC-20 Token in Remix IDE
 * Author       : Rutuja Bhalekar
 * Roll No      : 3
 * Token        : College Token (CLG), 18 decimals
 * Network      : Sepolia Testnet (via MetaMask)
 * ============================================================
 */

contract MyERC20Token {

    string public name = "College Token";
    string public symbol = "CLG";
    uint8 public decimals = 18;
    uint256 public totalSupply;

    // balanceOf(address) getter is generated by this public mapping
    mapping(address => uint256) public balanceOf;

    // allowance(owner, spender) getter is generated by this public mapping
    mapping(address => mapping(address => uint256)) public allowance;

    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);

    // The deployer receives the entire initial supply.
    // Example: initialSupply = 1000000  ->  totalSupply = 1000000 * 10^18
    constructor(uint256 initialSupply) {
        totalSupply = initialSupply * 10 ** uint256(decimals);
        balanceOf[msg.sender] = totalSupply;
        emit Transfer(address(0), msg.sender, totalSupply);
    }

    // Transfer tokens from the caller's account to another address
    function transfer(address to, uint256 value) public returns (bool success) {
        require(balanceOf[msg.sender] >= value, "Insufficient balance");
        balanceOf[msg.sender] -= value;
        balanceOf[to] += value;
        emit Transfer(msg.sender, to, value);
        return true;
    }

    // Give another address permission to spend tokens on your behalf
    function approve(address spender, uint256 value) public returns (bool success) {
        allowance[msg.sender][spender] = value;
        emit Approval(msg.sender, spender, value);
        return true;
    }

    // Transfer tokens using an approved allowance
    function transferFrom(address from, address to, uint256 value) public returns (bool success) {
        require(balanceOf[from] >= value, "Insufficient balance");
        require(allowance[from][msg.sender] >= value, "Allowance exceeded");
        allowance[from][msg.sender] -= value;
        balanceOf[from] -= value;
        balanceOf[to] += value;
        emit Transfer(from, to, value);
        return true;
    }
}
```

Deployment details (verified on Sepolia): Contract address: 0x63617493DED1f68501e9d45E4CF522032b427C0F; Deployer (MetaMask account "Rutuja

Bhalekar"): 0xd682b97bd00f4de6ba5d19b4181377b3acf5f74c; Deployment transaction: 0xaaacf57721642eeaee4a2ea005b03176ed6dc9c02ba07885f525dcf77465242b; Initial supply argument: 1000000 (totalSupply = 1000000000000000000000000); Transfer of 100 CLG to Account 2 (0xe17eAC4Fd4CeA7955F644337AF15Daa6A1A33740): 0x4193ff0e66c5a37b332646b7b5ec5bf6cfa35fc379c5a937b2a2da17fa13e2e9.

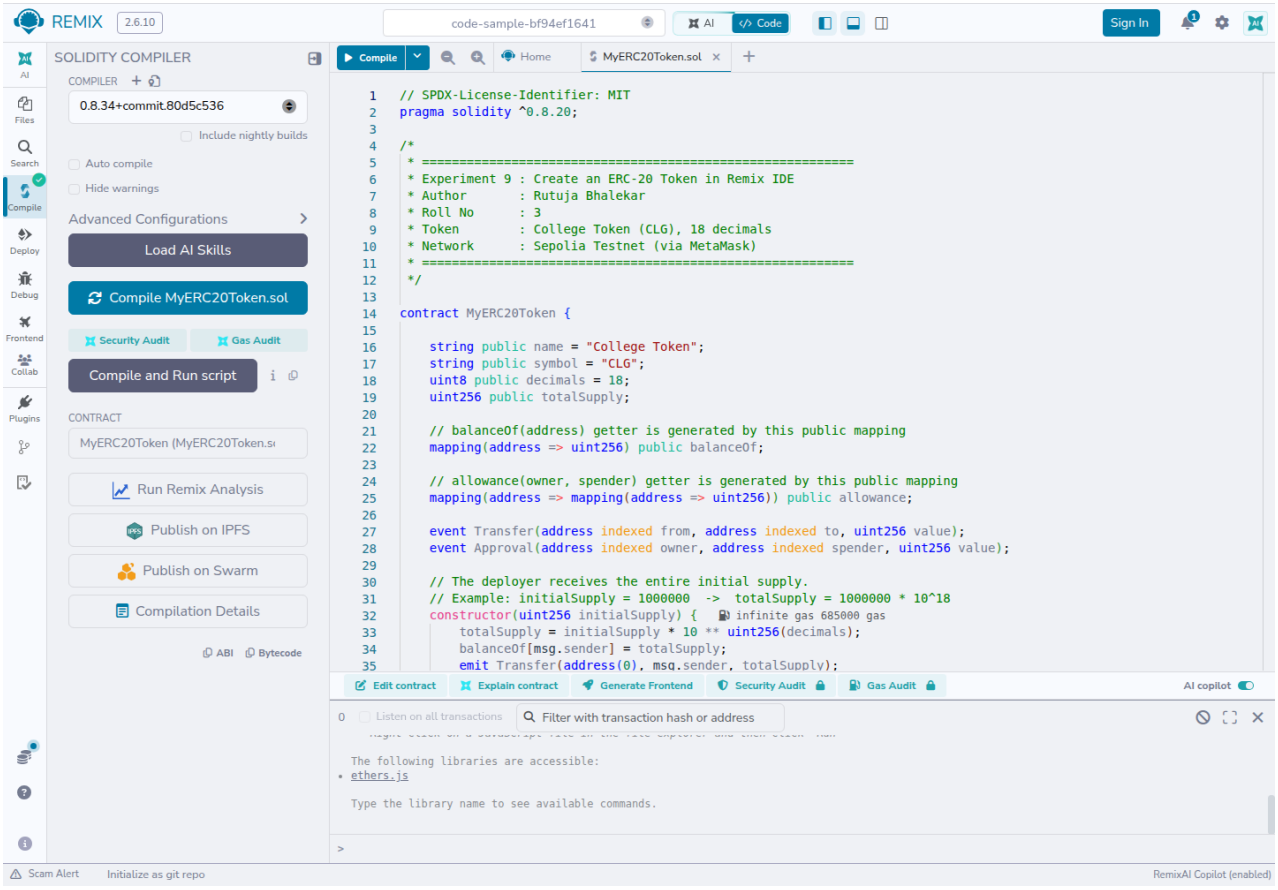


Fig. 1. Creating the MyERC20Token contract and compiling it in Remix IDE

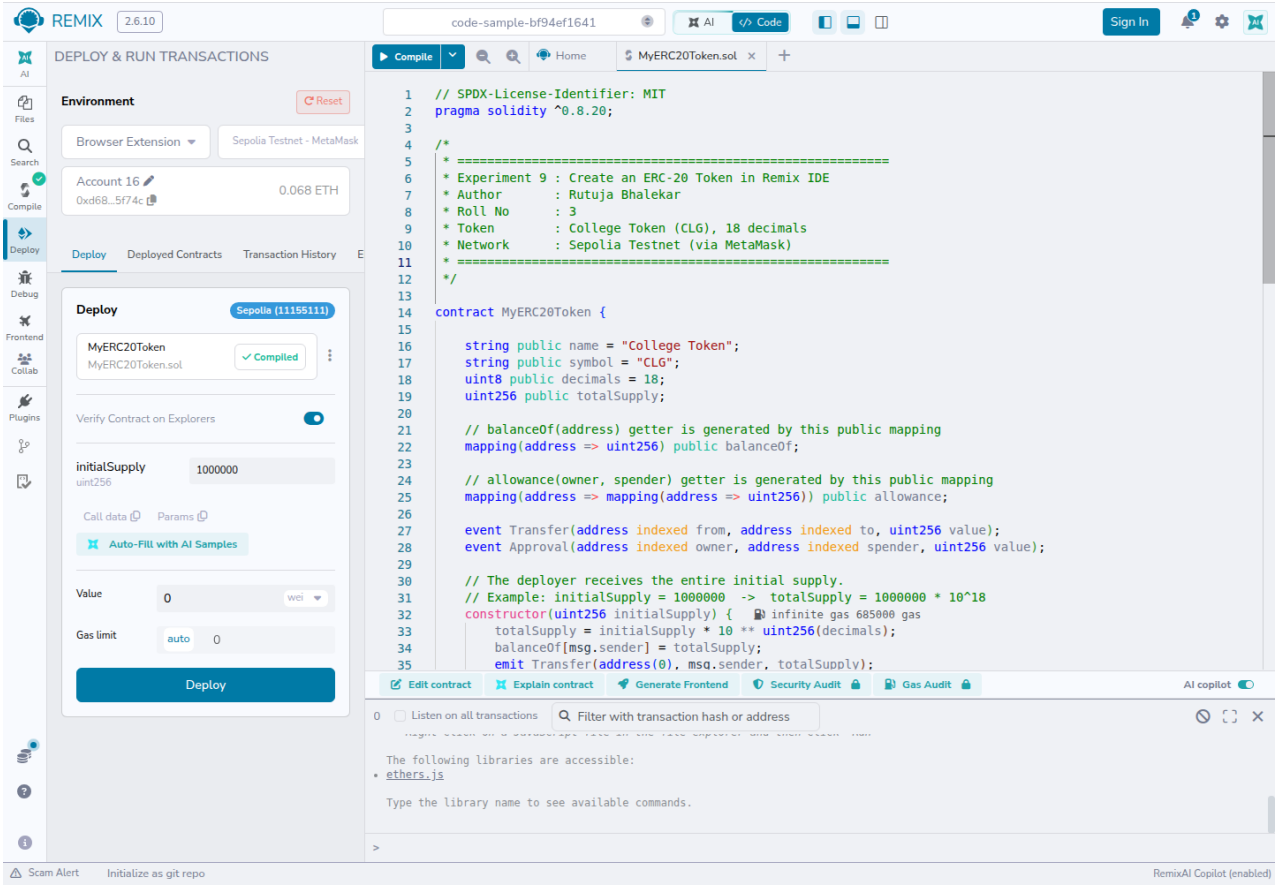


Fig. 2. Connecting MetaMask - Deploy panel on Sepolia Testnet with initialSupply 1000000

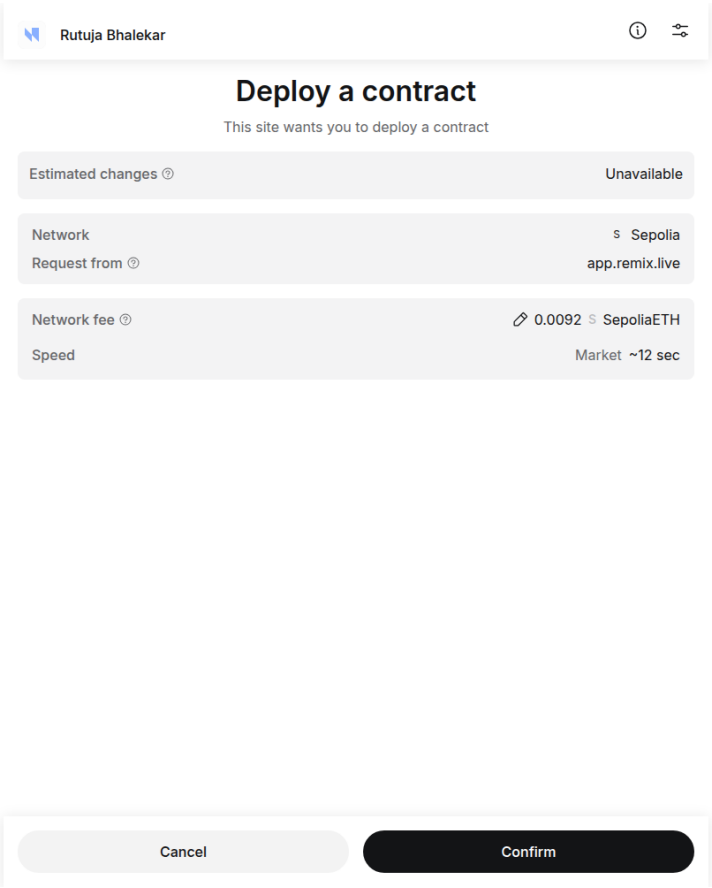


Fig. 3. Confirming the token contract deployment transaction via MetaMask

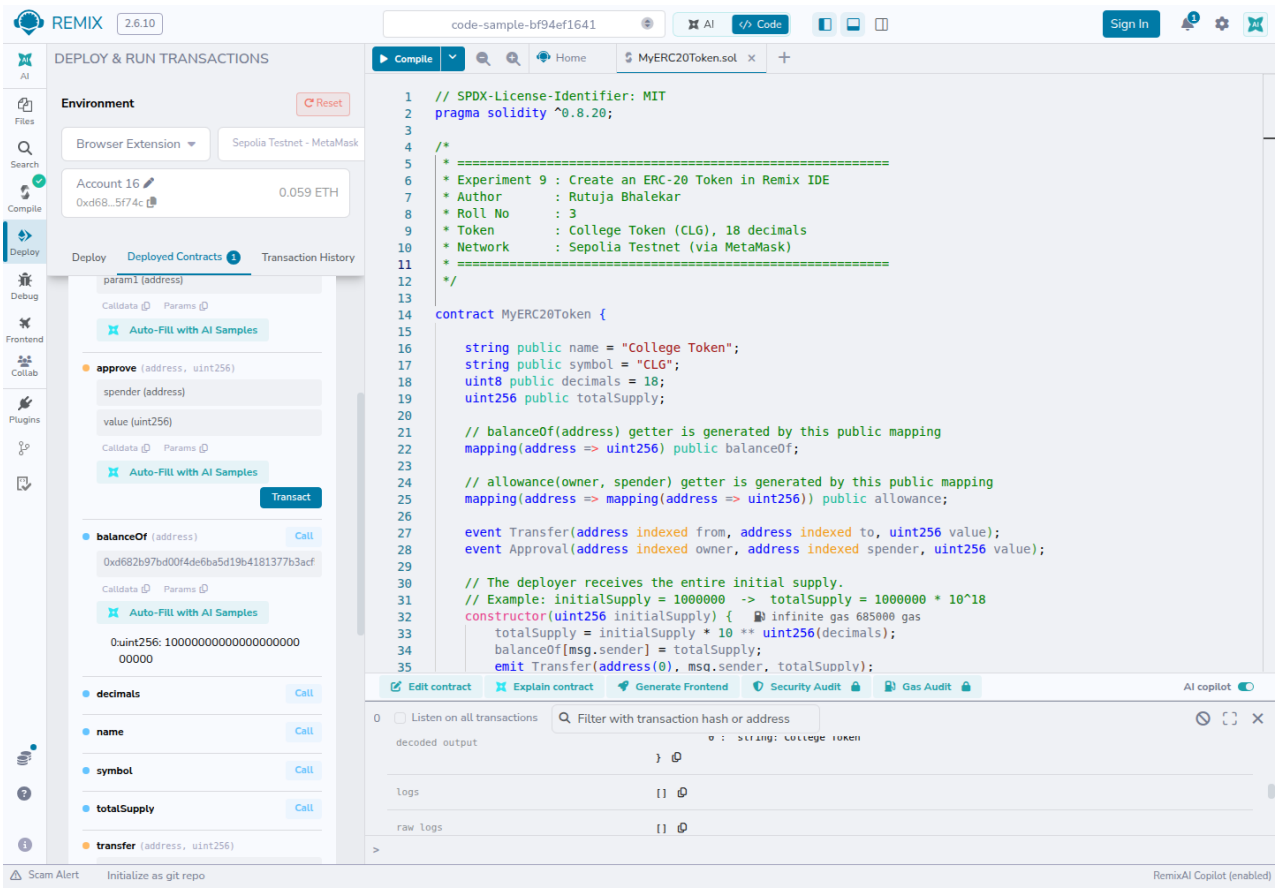


Fig. 4. Deployed contract - reading name, symbol, decimals, totalSupply and balanceOf

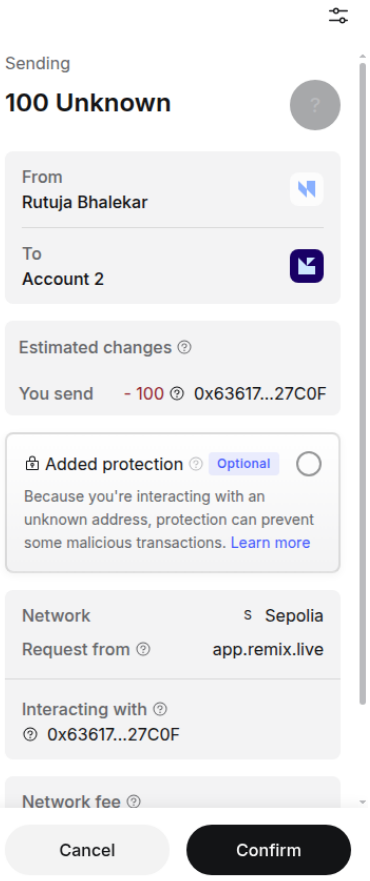


Fig. 5. Transferring tokens to another account - MetaMask confirmation

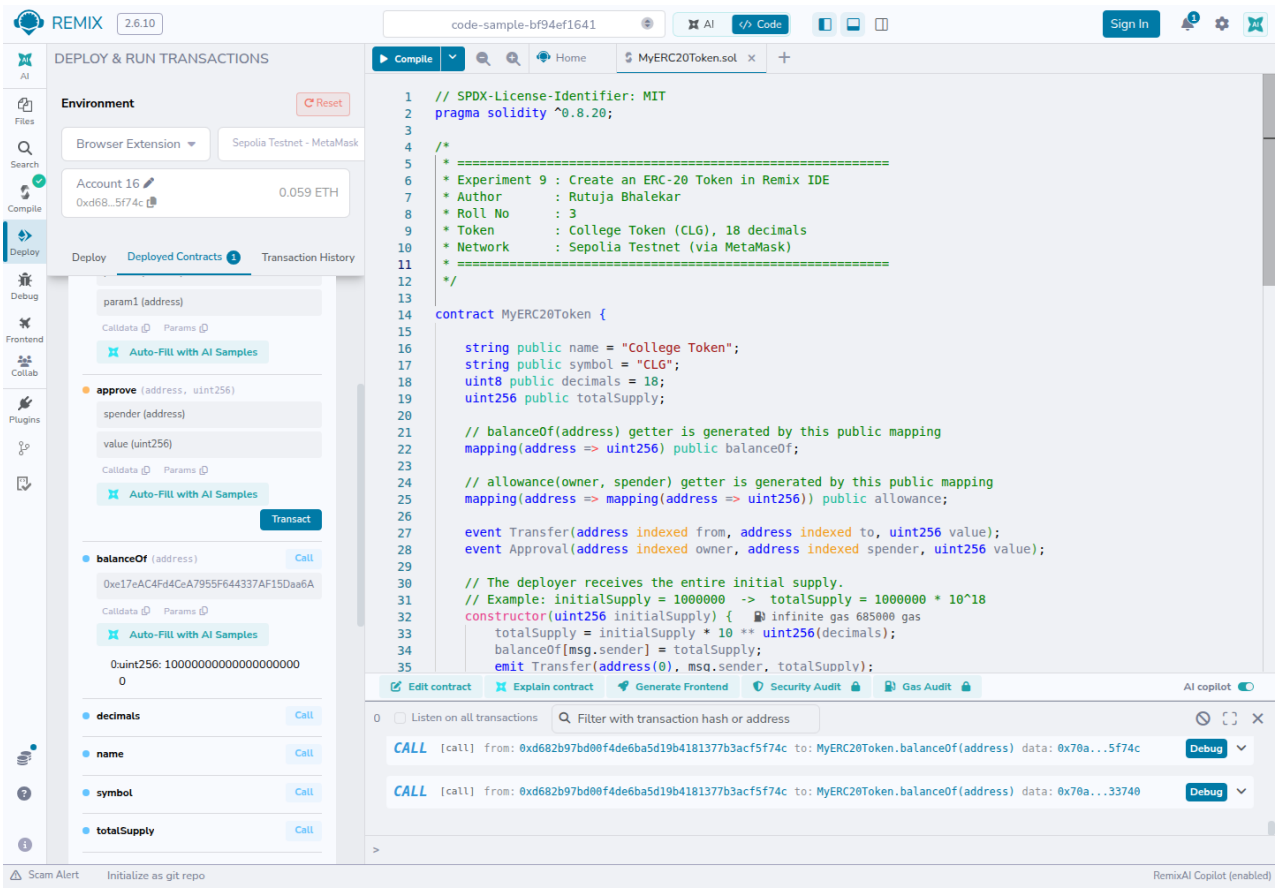


Fig. 6. Transfer result - balanceOf of the receiving account showing 100 CLG

CONCLUSION

Thus, the ERC-20 token was successfully created using Solidity in Remix IDE and deployed on the Ethereum Sepolia testnet using MetaMask. The token contract was successfully mined and assigned an initial supply of 1,000,000 College Tokens (CLG) with 18 decimals. A transfer of 100 CLG to a second account was also executed and verified through balanceOf. The experiment demonstrated the creation, deployment, and basic interaction with an ERC-20 smart contract, providing practical understanding of token standards, MetaMask wallet integration, blockchain transactions, and gas fees.